

ESP32-S3 Production Firmware

Corrected MQTT Sequence following EMQX Broker Best Practices

ETERNITY / ONEOPS Platform — companion to the *Unified Technical Specification v1.0*

Revised 2026-06-08

1 Scope and what was corrected

The specification’s end-to-end sequence (§5.6/§5.7) begins at **publish reading** and treats the broker as a dumb pipe. That hides the parts of the firmware contract that actually decide field reliability: *how the device authenticates, how presence is tracked, how sessions and commands survive a flaky link, and how firmware is delivered safely*. This document redraws the lifecycle of a single production device against **EMQX** broker semantics (MQTT 5.0) and marks every correction.

Area	Original spec	Corrected — EMQX best practice
Transport & auth	shared / implicit; “returns mTLS cert” but no handshake shown	one-time factory <i>bootstrap</i> token over server-auth TLS, immediately rotated to per-device X.509 client certificate ; steady state is mutual TLS 1.3 verified by EMQX.
Client identity	“initial <code>device_id</code> ”	stable, unique <code>mqtt_client_id</code> bound to the certificate CN; duplicate-ID takeover rejected.
Authorization	not stated	EMQX ACL : a device may only publish/subscribe under <code>devices/{id}/#</code> and <code>sensors/{sid}/raw</code> ; cross-device spoofing denied.
Session	not stated	MQTT 5.0 <code>clean_start=0</code> + Session-Expiry , so QoS 1 commands queued during a brief drop are re-delivered on reconnect.
Presence (LWT)	status is “LWT topic”, retain unspecified	Will published QoS 1, retain=1 ; matching birth message on (re)connect clears it, so late subscribers always read correct state.
Config delivery	plain publish	<code>devices/{id}/config</code> published retain=1, QoS 1 ⇒ device gets current config the instant it subscribes.
Heartbeat	QoS 0, retain=0 ✓	kept QoS 0; <code>keepalive=60 s</code> , EMQX dead-peer detection at 1.5× <code>keepalive</code> .
Sensor reading	QoS 1, retain=0 ✓	kept QoS 1 (PUBACK); ingest consumes via an EMQX shared subscription (<code>\$share/ingest/...</code>) for horizontal scale + back-pressure.
OTA	“upgrade to 1.4.2 from <code>artefact_uri</code> ”	signed artefact (SHA-256 + secure-boot), A/B partition, out-of-band HTTPS fetch, <code>ota/progress</code> telemetry, automatic rollback , audit row in <code>device_firmware_history</code> .
Abuse control	none	EMQX flapping detection + per-client message/bandwidth rate limits.

Legend. $\longrightarrow$ solid = MQTT control packet / synchronous call $\dashrightarrow$ dashed = broker delivery / asynchronous event $\dashrightarrow$ dotted = return / acknowledgement.

2 Phase A — Zero-touch provisioning & secure session bring-up

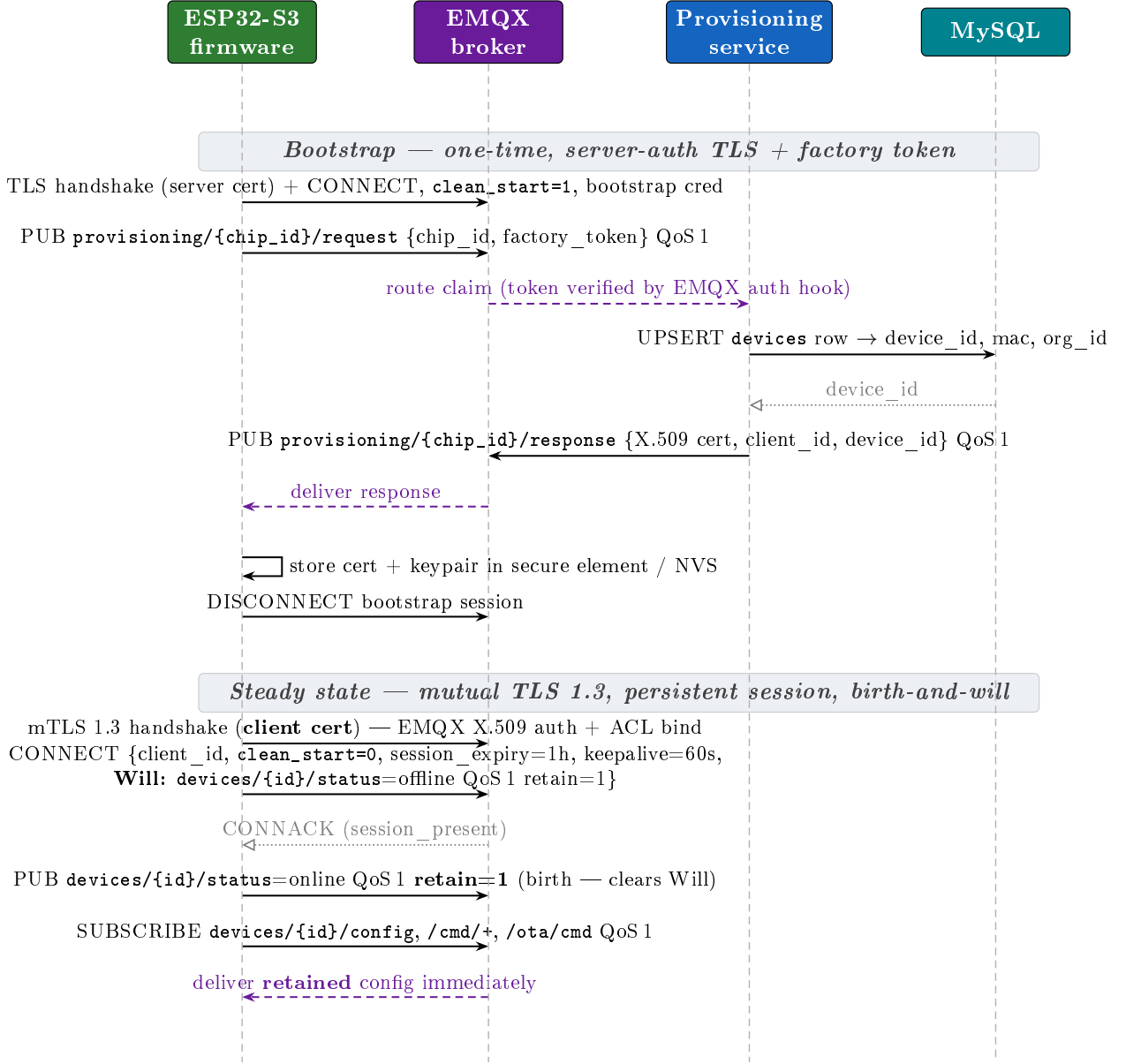


Figure 1: Phase A — provisioning rotates a factory token into a per-device certificate, then the production session is established with mutual TLS, a persistent MQTT 5.0 session, and a retained will/birth pair so presence is never stale.

3 Phase B — Steady-state telemetry (corrected ingestion)

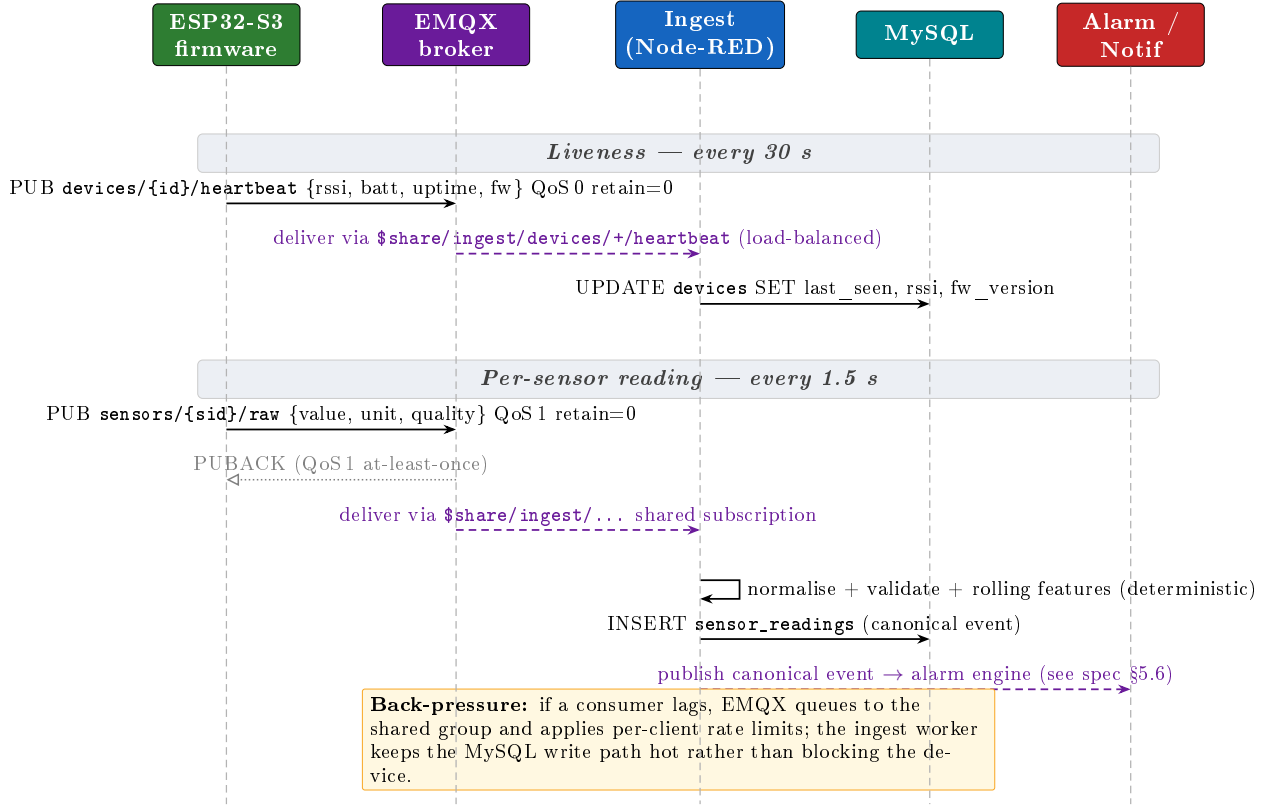


Figure 2: Phase B — heartbeats stay QoS 0; sensor readings stay QoS 1 with PUBACK. The single correction that matters for scale is consuming through an EMQX *shared subscription*, which load-balances across ingest replicas and gives the broker a place to absorb back-pressure.

4 Phase C — Signed OTA update with A/B rollback

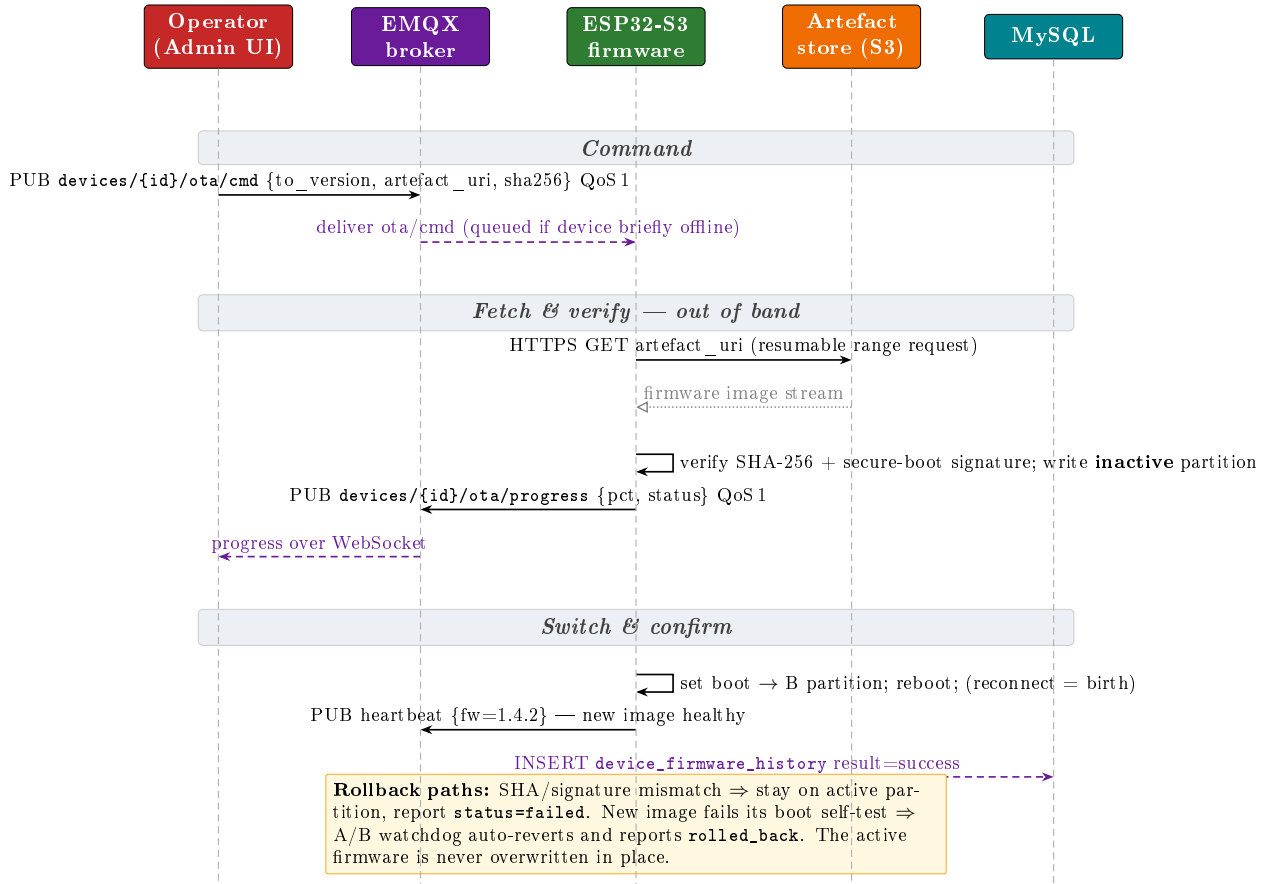


Figure 3: Phase C — OTA is delivered as a signed artefact fetched out of band over HTTPS, flashed to the inactive A/B slot, and only promoted after verification; the firmware version reported in the next heartbeat closes the audit loop.

5 Phase D — Ungraceful disconnect (Last Will & Testament)

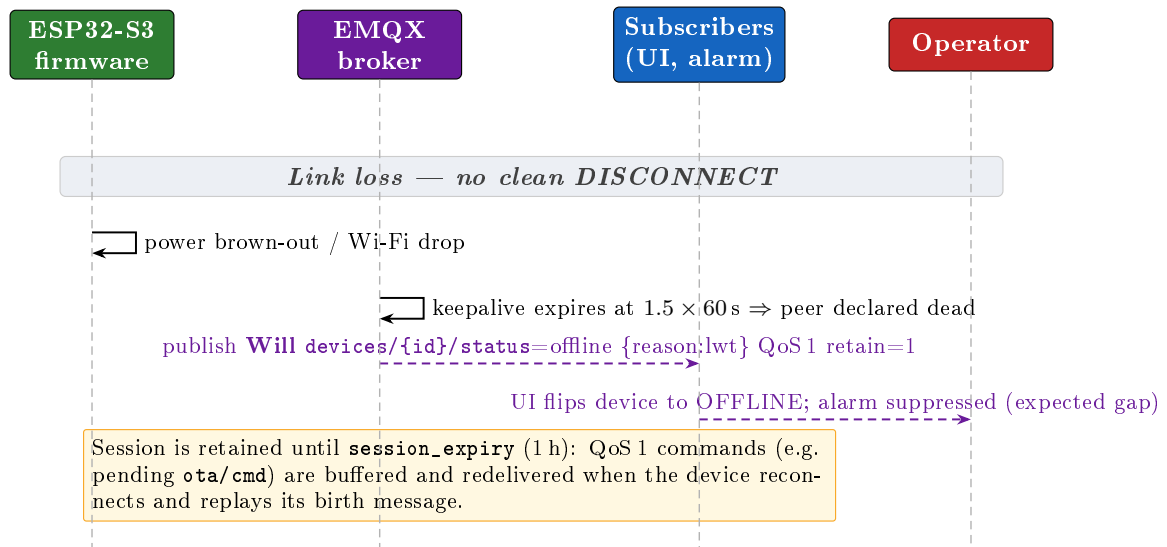


Figure 4: Phase D — because the Will is registered at CONNECT with retain=1, presence stays correct without any device cooperation; the retained session keeps queued commands alive across the outage.

6 Per-product telemetry — one envelope, three payload profiles

The phases above are deliberately product-agnostic because every device speaks the *same* firmware contract. Temperature is transmitted by all three product lines, but it is only *one channel* inside a product-specific set. A device emits **one sensors/{sid}/raw publish per channel per cycle**; the broker and ingest path are identical, and the meaning of each `sensor_id` is resolved by the polymorphic *CTI* model (`sensors.host_fk` $\in$ {t,c,b} $\rightarrow$ exactly one of the three `*_sensor_specs` tables). What actually differs per product is the *channel set*, units, and cadence.

Product line (host)	Cadence		Channels emitted as sensors/{sid}/raw
Refrigeration Data Logger (carbonbox, host=c)	temp 30–60 s;	door on-change	temp_c (°C); door_state (bool 0/1 — <i>event</i> , not a scalar measurement). Cold-chain min/max only.
BloodBOX (host=b)	temp/RH 10–30 s;GPS 1–5 s in transit;impact event-driven		temp_c, rh_pct (humidity), batt_pct, impact_g (shock), gps (lat/lng — <i>compound</i>), baro_alt_m (floor/altitude). Medical transit; richest payload.
Transformer Monitoring (eternity, host=t)	thermal 30–60 s;	DGA minutes–hours	oil_temp_c, ambient_temp_c, dga_h2_ppm (dissolved hydrogen), moisture_pct, oil_level_pct, load_pct. Multi-channel DGA + thermal.

The deltas the generic envelope must absorb: `door_state` and `impact_g` are boolean/event channels mapped as `value` $\in$ {0,1} / `unit=event`; `gps` is a compound channel (sent as two readings `lat`, `lng` or one structured value); cadences span three orders of magnitude (GPS seconds vs. DGA hours). None of this changes the topic or QoS — only the channel set and the spec table the ingest worker joins to.

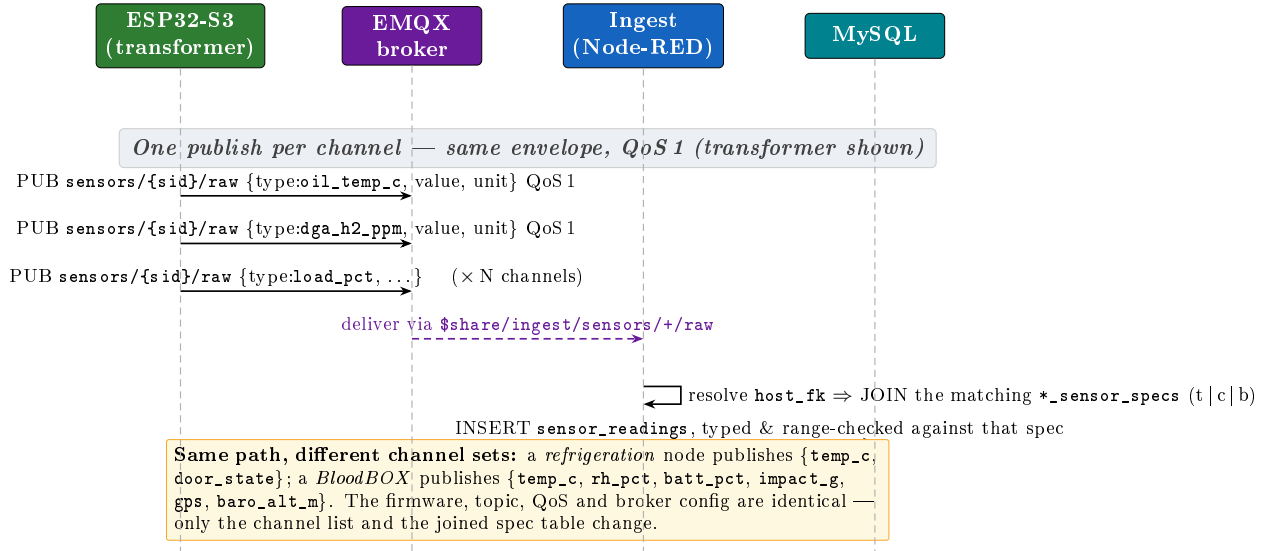


Figure 5: Phase B (per-product) — each device fans out one reading per channel over the single `sensors/{sid}/raw` envelope; the ingest worker resolves the polymorphic host FK and joins the correct per-type spec table, so a 6-channel transformer and a 2-channel fridge share one pipeline.

7 MQTT topic & QoS contract (corrected)

Topic	Dir	QoS	Retain	Notes
provisioning/{chip_id}/request	D→B	1	0	bootstrap listener only
provisioning/{chip_id}/response	B→D	1	0	returns X.509 cert + client_id
devices/{id}/status	D→B	1	1	Will <i>and</i> birth on this topic
devices/{id}/heartbeat	D→B	0	0	30s; liveness & fw_version
sensors/{sid}/raw	D→B	1	0	consumed via \$share/ingest/
devices/{id}/config	B→D	1	1	retained so it is read on subscribe
devices/{id}/cmd/{op}	B→D	1	0	reboot / calibrate / set-threshold
devices/{id}/ota/cmd	B→D	1	0	signed artefact descriptor
devices/{id}/ota/progress	D→B	1	0	percent + status

EMQX configuration checklist.

- Dedicated TLS listeners: a *bootstrap* listener (server-auth, token) and a *production* listener (mutual TLS, `verify=verify_peer, fail_if_no_peer_cert=true`).
- Authentication chain: X.509 CN → `client_id`; reject mismatched or duplicate client IDs.
- Authorization (ACL) pinned to `${clientid}` so each device is boxed into its own `devices/{id}/#` and `sensors/{sid}/raw`.
- Shared subscriptions (`$share/ingest/...`) for every server-side consumer; never a single static subscriber.
- Flapping detection + per-client rate & bandwidth limits enabled.
- Session expiry sized to the longest tolerable command-delivery gap (here 1 h).